

TP2 — Sistema de Reserva de Passagens de Ônibus

API RESTful com Spring Boot (CRUD em memória)

Júlia Caroline Tallarico

TP2 — Sistema de Reserva de Passagens de Ônibus

Aluna: Júlia Caroline Tallarico **Disciplina:** Desenvolvimento de Serviços com Spring Boot

Introdução

Neste trabalho eu construí uma API RESTful com Spring Boot para gerenciar a reserva de passagens de ônibus. A ideia principal era praticar as operações de CRUD guardando os dados em memória (numa ArrayList, sem banco de dados) e, ao mesmo tempo, manter a separação de responsabilidades em camadas: controllers, services, dtos e models.

Conforme o enunciado, usei apenas **Spring Web**, **Spring Boot DevTools** e **Lombok**, sem nenhuma biblioteca de validação. Nas seções abaixo eu vou seguindo os exercícios e explicando o que fiz em cada parte, com os trechos de código mais importantes e exemplos das requisições que testei.

O projeto foi feito com Java 17, Spring Boot 3.3.2 e Maven.

Exercício 1 — Criação dos projetos

Criei o projeto principal pela interface web do **Spring Initializr** (<https://start.spring.io>), escolhendo **Maven** como ferramenta de build. Prefiro Maven porque já tinha usado antes e acho o pom.xml mais fácil de ler, além de a maioria dos exemplos da documentação do Spring vir em Maven. As configurações foram:

- Project: Maven / Language: Java / Spring Boot: 3.3.2
- Group: br.com.juliatallarico — Artifact: reserva-passagens
- Java: 17
- Dependências: **Spring Web**, **Spring Boot DevTools** e **Lombok**

Para praticar a linha de comando, também gerei um **projeto secundário** com a **Spring Boot CLI**, no terminal:

```
spring init --build=maven --java-version=17 \
--dependencies=web,devtools,lombok \
--group-id=com.example --artifact-id=demo demo
```

As duas pastas (reserva-passagens e projeto-cli) estão incluídas no .zip final. O projeto com a implementação do TP é o reserva-passagens; o projeto-cli é só o exercício da CLI.

Exercício 2 — Organização em pacotes

Organizei o código nos quatro pacotes pedidos, cada um com a sua responsabilidade:

```
br.com.juliatallarico.reservapassagens
├── ReservaPassagensApplication.java (classe principal)
├── controllers
│   └── PassagemController.java      (rotas / HTTP)
├── services
│   └── PassagemService.java         (regras de negócio + lista em memória)
├── dtos
│   ├── PassagemRequestDTO.java      (dados que entram)
│   └── PassagemResponseDTO.java      (dados que saem)
└── models
    └── Passagem.java                (a entidade)
```

A ideia foi deixar o controller só com o “lado web”, o service com a lógica e os dados, os DTOs como contrato de entrada/saída e o model representando a passagem.

Exercício 3 — A classe Passagem (model)

No pacote models criei a classe Passagem usando as anotações do Lombok, que geram automaticamente os getters, setters, toString, e os construtores — assim eu não preciso escrever isso tudo na mão:

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Passagem {
    private Long id;
    private String passageiro;
    private Integer assento;
    private String origem;
    private String destino;
    private LocalDate data;
    private String status;
}
```

Exercício 4 — Os DTOs

No pacote dtos criei dois objetos. O PassagemRequestDTO representa os dados que chegam numa criação/atualização (não tem id, porque quem gera o id é o service) e o

PassagemResponseDTO representa os dados que a API devolve (esse sim com o id). Como o enunciado pediu, **não coloquei nenhuma anotação de validação**.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class PassagemRequestDTO {
    private String passageiro;
    private Integer assento;
    private String origem;
    private String destino;
    private LocalDate data;
    private String status;
}
```

O PassagemResponseDTO é igual, mas tem também o campo id.

Exercícios 5 e 6 — O service e os dados iniciais

No pacote services criei a classe PassagemService com @Service. Dentro dela declarei a lista em memória e o contador de ids:

```
private List<Passagem> passagens = new ArrayList<>();
private Long idCounter = 1L;
```

No construtor eu já instancio 3 passagens, atribuindo o id com o idCounter e incrementando ele a cada uma, pra a aplicação já subir com alguns dados pra testar:

```
public PassagemService() {
    passagens.add(new Passagem(idCounter++, "Ana Souza", 1, "Sao Paulo",
        "Rio de Janeiro", LocalDate.of(2026, 6, 10), "CONFIRMADA"));
    passagens.add(new Passagem(idCounter++, "Bruno Lima", 2, "Campinas",
        "Curitiba", LocalDate.of(2026, 6, 11), "CONFIRMADA"));
    passagens.add(new Passagem(idCounter++, "Carla Dias", 3, "Belo Horizonte",
        "Vitoria", LocalDate.of(2026, 6, 12), "PENDENTE"));
}
```

Exercício 7 — Métodos de conversão

Para converter entre o DTO e o model, criei dois métodos auxiliares dentro do service: um que transforma o PassagemRequestDTO em Passagem e outro que transforma a Passagem em PassagemResponseDTO:

```
private Passagem toEntity(PassagemRequestDTO dto) {
    Passagem p = new Passagem();
    p.setPassageiro(dto.getPassageiro());
    p.setAssento(dto.getAssento());
}
```

```

    p.setOrigem(dto.getOrigem());
    p.setDestino(dto.getDestino());
    p.setData(dto.getData());
    p.setStatus(dto.getStatus());
    return p;
}

private PassagemResponseDTO toResponse(Passagem p) {
    return new PassagemResponseDTO(p.getId(), p.getPassageiro(),
        p.getAssento(),
        p.getOrigem(), p.getDestino(), p.getData(), p.getStatus());
}

```

Deixei esses métodos como private no service porque é só ali que eu uso eles.

Exercício 8 — O controller e o GET da lista

No pacote controllers criei o PassagemController com @RestController e @RequestMapping("/passagens"), injetando o service pelo construtor. O primeiro endpoint é o GET /passagens, que devolve a lista completa:

```

@RestController
@RequestMapping("/passagens")
public class PassagemController {

    private final PassagemService service;

    public PassagemController(PassagemService service) {
        this.service = service;
    }

    @GetMapping
    public List<PassagemResponseDTO> listar() {
        return service.listarTodas();
    }
}

```

No service, o listarTodas percorre a lista convertendo cada passagem para o DTO de resposta:

```

public List<PassagemResponseDTO> listarTodas() {
    return passagens.stream().map(this::toResponse).toList();
}

```

Exercícios 9 e 10 — Criação (POST) com verificação de assento

O POST /passagens recebe um PassagemRequestDTO no corpo e devolve **201 Created**:

@PostMapping

```
public ResponseEntity<PassagemResponseDTO> criar(@RequestBody
PassagemRequestDTO dto) {
    PassagemResponseDTO criada = service.criar(dto);
    return ResponseEntity.status(HttpStatus.CREATED).body(criada);
}
```

No service, antes de adicionar a passagem na lista, eu verifico se o assento já está ocupado. Se estiver, lanço um `ResponseStatusException` com status **400 (BAD_REQUEST)**. Se estiver livre, eu gero o id com o `idCounter`, incremento e adiciono:

```
public PassagemResponseDTO criar(PassagemRequestDTO dto) {
    boolean assentoOcupado = passagens.stream()
        .anyMatch(p -> p.getAssento().equals(dto.getAssento()));
    if (assentoOcupado) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
            "O assento " + dto.getAssento() + " ja esta ocupado.");
    }
    Passagem nova = toEntity(dto);
    nova.setId(idCounter++);
    passagens.add(nova);
    return toResponse(nova);
}
```

Exercício 11 — Buscar por id (GET /passagens/{id})

Esse endpoint busca uma passagem pelo id. No service eu uso `stream().filter().findFirst()` e, se não achar, lanço **404 (NOT_FOUND)**. Como essa busca é usada em vários lugares (no GET, no PUT e no DELETE), coloquei ela num método auxiliar `encontrarOuFalhar`:

@GetMapping("/{id}")

```
public PassagemResponseDTO buscarPorId(@PathVariable Long id) {
    return service.buscarPorId(id);
}
```

```
private Passagem encontrarOuFalhar(Long id) {
```

```
    return passagens.stream()
        .filter(p -> p.getId().equals(id))
        .findFirst()
        .orElseThrow(() -> new
ResponseStatusException(HttpStatus.NOT_FOUND,
    "Passagem com id " + id + " não encontrada."));
}
```

Exercício 12 — Atualizar (PUT /passagens/{id})

Usei a anotação `@PutMapping`. No service eu busco a passagem pelo id (e, se não existir, cai no 404 do método auxiliar) e atualizo os atributos com os dados do DTO, retornando o objeto já atualizado:

```
@PutMapping("/{id}")
public PassagemResponseDTO atualizar(@PathVariable Long id, @RequestBody
PassagemRequestDTO dto) {
    return service.atualizar(id, dto);
}

public PassagemResponseDTO atualizar(Long id, PassagemRequestDTO dto) {
    Passagem p = encontrarOuFalhar(id);
    p.setPassageiro(dto.getPassageiro());
    p.setAssento(dto.getAssento());
    p.setOrigem(dto.getOrigem());
    p.setDestino(dto.getDestino());
    p.setData(dto.getData());
    p.setStatus(dto.getStatus());
    return toResponse(p);
}
```

Exercício 13 — Remover (DELETE /passagens/{id})

Usei `@DeleteMapping` e o endpoint retorna **204 (No Content)**. No service eu removo da lista com `removeIf`. Aproveitei o retorno do `removeIf` (que é `true/false`) para lançar 404 caso o id não exista:

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deletar(@PathVariable Long id) {
    service.deletar(id);
    return ResponseEntity.noContent().build();
}

public void deletar(Long id) {
    boolean removeu = passagens.removeIf(p -> p.getId().equals(id));
    if (!removeu) {
        throw new RuntimeException(HttpStatus.NOT_FOUND,
            "Passagem com id " + id + " nao encontrada.");
    }
}
```

Exercício 14 — Busca por destino (GET /passagens/busca)

Esse endpoint recebe o destino por `@RequestParam` e devolve só as passagens daquele destino. Um detalhe que tive que pensar: como existe também o `/passagens/{id}`, eu

declarei o /busca separado — e o Spring dá preferência ao caminho literal “busca” em vez de tentar encaixar como {id}, então não dá conflito.

```
@GetMapping("/busca")
public List<PassagemResponseDTO> buscarPorDestino(@RequestParam String
destino) {
    return service.buscarPorDestino(destino);
}

public List<PassagemResponseDTO> buscarPorDestino(String destino) {
    return passagens.stream()
        .filter(p -> p.getDestino().equalsIgnoreCase(destino))
        .map(this::toResponse)
        .toList();
}
```

Exercício 15 — Garantia de 404

Como o GET por id, o PUT e o DELETE todos passam pela mesma verificação (o encontrarOuFalhar ou o retorno do removeIf), qualquer operação que use um id que não está na lista lança o erro **404 (Not Found)** com uma mensagem explicando. Centralizar essa checagem num método só evitou repetir código e garante o comportamento igual nos três casos.

Exercício 16 — Testes

Subi a aplicação com mvn spring-boot:run e testei todos os fluxos no Postman. Abaixo estão alguns dos testes que fiz, com a resposta de cada um.

GET /passagens — lista as 3 passagens iniciais:

```
[
  {"id":1,"passageiro":"Ana Souza","assento":1,"origem":"Sao Paulo","destino":"Rio
de Janeiro","data":"2026-06-10","status":"CONFIRMADA"},
  {"id":2,"passageiro":"Bruno
Lima","assento":2,"origem":"Campinas","destino":"Curitiba","data":"2026-06-
11","status":"CONFIRMADA"},
  {"id":3,"passageiro":"Carla Dias","assento":3,"origem":"Belo
Horizonte","destino":"Vitoria","data":"2026-06-12","status":"PENDENTE"}
]
```

POST /passagens — criando uma passagem nova (retorna 201):

Corpo enviado:

```
{"passageiro":"Diego Alves","assento":4,"origem":"Santos","destino":"Sao
Paulo","data":"2026-06-15","status":"CONFIRMADA"}
```

Resposta:

```
{ "id":4, "passageiro":"Diego Alves", "assento":4, "origem":"Santos", "destino":"Sao Paulo", "data":"2026-06-15", "status":"CONFIRMADA" }
```

POST /passagens com um assento já ocupado (assento 1) — retorna 400:

```
{ "status":400, "error":"Bad Request", "message":"O assento 1 ja esta ocupado." }
```

GET /passagens/busca?destino=Curitiba — filtra por destino:

```
[  
  { "id":2, "passageiro":"Bruno Lima", "assento":2, "origem":"Campinas", "destino":"Curitiba", "data":"2026-06-11", "status":"CONFIRMADA" }  
]
```

GET /passagens/99 — id inexistente, retorna 404:

```
{ "status":404, "error":"Not Found", "message":"Passagem com id 99 nao encontrada." }
```

DELETE /passagens/2 — remove e retorna 204 (No Content), sem corpo.

Todos os fluxos (GET, POST, PUT, DELETE e busca) funcionaram como esperado.

Conclusão

Com esse TP eu consegui montar uma API REST completa com Spring Boot mantendo a separação em camadas, que pra mim foi o ponto mais importante: deixar cada parte com a sua responsabilidade (controller, service, dto e model) faz o código ficar bem mais organizado e fácil de mexer depois. O Lombok ajudou bastante a deixar as classes enxutas, e usar a lista em memória com o contador de ids foi uma forma simples de praticar o CRUD antes de partir pra um banco de dados de verdade.